26

DEEP DIVE 26 · పని తనకే పని ఇస్తే

Design a Thread Pool

> ThreadPoolExecutor · goroutine pools · libuv · ప్రతి web server లోపల – "threads ని ముందే తయారు చేసి, పనులు queue lo పెడదాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – మీరు **max 50** అని రాస్తారు, మీకు **2 వస్తాయి**, మరియు p99 **19.9 సెకన్లు** అవుతుంది. రెండోది – **7 పనులు బాగా నడుస్తాయి, 8వది system ని శాశ్వతంగా ఆపేస్తుంది**. మూడోది – నిండినప్పుడు ఉన్న మూడు దారుల్లో ఒకటి **57% పనిని పోగొడుతుంది**, ఇంకొకటి **59% సమయం producer ని ఆపేస్తుంది**.

తెలుగు + ENGLISH

YASWANTH · DEEP DIVE 26

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి **ప్రతి output, ప్రతి సంఖ్య నిజంగా `node` lo run చేసినదే.**

DEEP DIVE 24 (CONNECTION POOL) చదివారా? — రెండూ POOLS, కానీ ఒకటి మాత్రమే ఆగిపోగలదు

DD 24 lo పరిమిత వనరులు, గడువులు, తిరస్కరణ చూశాం. అవన్నీ ఇక్కడా వర్తిస్తాయి, కాబట్టి వాటిని మళ్ళీ మొదటి నుంచి వివరించను.

కానీ thread pool కి connection pool కి ఒక **ప్రాథమికమైన** తేడా ఉంది, మరియు అది §7 మొత్తానికి కారణం:

ఒక query ఇంకొక query ని అడగలేదు. ఒక పని ఇంకొక పనిని అడగగలదు.

Connection pool lo, ఒక connection తీసుకున్నవాడు ఆ connection ని వాడతాడు, తిరిగి ఇస్తాడు. అతను **pool ని మళ్ళీ అడగడు**.

Thread pool lo, ఒక పని లోపల `pool.submit(...)` రాయడం **పూర్తిగా సహజం** — మరియు ఆ ఉప-పని కోసం ఎదురుచూస్తే, మీ thread ఇంకా పట్టుకునే ఉంది. అందరూ అలా చేస్తే **ఎవరికీ thread మిగలదు**, మరియు pool **శాశ్వతంగా ఆగిపోతుంది**. §7 lo అది 7 పనుల దగ్గర బాగానే ఉంది, 8వ దగ్గర పూర్తిగా చచ్చిపోయింది.

ఇది ఒక పునరావృత్తి (recursion) సమస్య, ఒక వనరు సమస్య కాదు. DD 24 lo దీనికి సమానమైనది లేదు.

విషయ సూచిక (Table of Contents)

Part 1 — సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి — threads ఎందుకు ముందే తయారు చేస్తాం?
2. Clarifying questions — ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 — మొదటి విరుపు: వాడబడని `max`

3. Step — core, max, queue · మూడు నియమాలు
4. **మొదటి విరుపు** — 50 అడిగారు, 2 వచ్చాయి
5. Step — queue ని **చిన్నది** చేయడం

Part 3 — రెండో విరుపు: పని తనకే పని ఇస్తే

6. Step — ఒక పని లోపల ఇంకొక పని
7. **రెండో విరుపు** — 7 బాగుంది, 8 శాశ్వతంగా ఆగింది
8. Step — bulkhead · వేరే పనికి వేరే pool

Part 4 — మూడో విరుపు: నిండినప్పుడు

9. Step — నిండితే ఏమి చెయ్యాలి?
10. **మూడో విరుపు** — మూడు దారులు, మూడింటికీ ఖరీదు

11. Step – నిర్ణయాన్ని బయటికి తీయడం

Part 5 – పూర్తి system

12. Step – ఎన్ని threads? (సూత్రం, మరియు దాన్ని కొలిచి చూడటం)
13. మొత్తం code · 8.5 లక్షల tasks · **mutation testing**

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటితో చెప్పాల్సిన English script
16. Follow-ups – work stealing, ప్రాధాన్యత, virtual threads
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

---

1. అడిగింది ఏమిటి – threads ఎందుకు ముందే తయారు చేస్తాం?

రెండు కారణాలు, మరియు DD 24 §1 లాగే **రెండోదే ముఖ్యమైనది**.

మొదటిది – thread తయారు చేయడం ఖరీదు. ఒక OS thread కి 1 MB stack, kernel lo ఒక నమోదు, మరియు scheduler lo ఒక ప్రవేశం కావాలి. అది సుమారు మిల్లీసెకను. ఒక్కో request కి ఒక thread తయారు చేస్తే, ఆ ఖర్చు ప్రతిసారీ చెల్లించాలి.

రెండోది – threads *ఉండటం* కూడా ఖరీదు. ఇదే అసలు విషయం.

> **THREAD POOL యొక్క అసలు పని – మరియు ఇది DD 24 తో ఒకటే**
>
> 10,000 threads తయారు చేయగలరు. కానీ మీ CPU కి **4 cores** మాత్రమే ఉన్నాయి. ఆ 10,000 threads **ఒకేసారి 4 మాత్రమే** నడుస్తాయి; మిగతా 9,996 ఎదురుచూస్తూ, **10 GB memory** తింటూ, మరియు OS ని వాటి మధ్య ఎడతెగకుండా మార్చేలా చేస్తూ ఉంటాయి.
>
> ఆ మార్పుకి (context switch) ఒక్కొక్కదానికీ కొన్ని మైక్రోసెకన్లు. 10,000 threads తో OS **పని చేయడం కంటే మారడానికే** ఎక్కువ సమయం ఖర్చు చేస్తుంది.
>
> కాబట్టి thread pool ఒక **cache కాదు, ఒక అడ్డుకట్ట** – సరిగ్గా DD 24 §1 lo చెప్పినట్టే. అది మీ machine ని మీ స్వంత traffic నుంచి కాపాడుతుంది.

కానీ ఇక్కడ ఒక **మూడో** విషయం ఉంది, మరియు అది connection pool కి లేనిది:

Connection pool కీ thread pool కీ ఒక ప్రాథమిక తేడా

Connection Pool

వాడేవాడు ఒక connection తీసుకుంటాడు,

వాడతాడు, తిరిగి ఇస్తాడు.

అతను **pool ని మళ్ళీ అడగడు**.

Thread Pool

ఒక పని లోపల **pool.submit()** రాయడం

పూర్తిగా సహజం.

అంటే pool **తనను తానే** అడగగలదు.

ఒక query ఇంకొక query ని అడగలేదు. **ఒక పని ఇంకొక పనిని అడగగలదు.**

ఆ ఉప-పని కోసం ఎదురుచూస్తే – మీ thread ఇంకా పట్టుకునే ఉంది.

అందరూ అలా చేస్తే, ఉప-పనులకి thread మిగలదు, మరియు అవి ఎప్పటికీ పూర్తి కావు.

అదే §7 – మరియు అది ఒక నెమ్మదితనం కాదు, ఒక **శాశ్వతమైన ఆగిపోవడం**.

---

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
పనులు CPU మీద లెక్కిస్తాయా, లేదా ఎదురుచూస్తాయా?	ఇదే thread సంఖ్యని నిర్ణయిస్తుంది – §12 lo 4 vs **44**
ఒక పని లోపల ఇంకొక పనిని submit చేస్తారా?	**అవును అంటే deadlock వస్తుంది.** ఇది అడగకపోతే §7 మిమ్మల్ని కొడుతుంది
Queue నిండితే ఏమి చెయ్యాలి – తిరస్కరించాలా, ఎదురుచూడాలా, వదిలేయాలా?	మూడూ సరైనవే, మూడింటికీ వేరే ఖరీదు. §10
పనులన్నీ ఒకే రకమేనా, లేక కొన్ని చాలా నెమ్మదా?	కలిపితే నెమ్మదైనవి మిగతా అన్నిటినీ ఆపేస్తాయి. §8
Shutdown అయినప్పుడు queue lo ఉన్న పని ఏమవుతుంది?	**మౌనంగా పోకూడదు** – §13 lo `shutdownNow` దాన్ని *తిరిగి ఇస్తుంది*
ఒక పని exception విసిరితే thread చచ్చిపోతుందా?	చావకూడదు. ఒక పని ఇంకొక పనిని చంపకూడదు
పనులకి ప్రాధాన్యతలు ఉన్నాయా?	ఉంటే starvation ఒక కొత్త సమస్య. §16

ఒక ప్రశ్న తప్పక అడగాలి – మరియు చాలామంది అడగరు

"ఒక పని లోపల ఇంకొక పనిని submit చేసి దాని కోసం ఎదురుచూస్తారా?"

ఇది అడిగితే interviewer కి వెంటనే తెలుస్తుంది – మీరు ఈ problem ని **నిజంగా ఎదుర్కొన్నారని**. ఎందుకంటే ఈ ప్రశ్న పుస్తకాల్లో ఉండదు; అది ఒకసారి production lo ఇది జరిగినవాళ్ళకే తడుతుంది.

మరియు సమాధానం "అవును" అయితే, మీ మొత్తం design మారుతుంది (§8).

Part 2 – మొదటి విరుపు: వాడబడని `max`

3. Step – core, max, queue · మూడు నియమాలు

ప్రతి నిజమైన thread pool కి మూడు సంఖ్యలు ఉంటాయి: `core` (ఎప్పుడూ ఉండే threads), `max` (గరిష్ఠం), మరియు `queueCap` (ఎదురుచూసే పనుల పరిమితి).

ఒక కొత్త పని వచ్చినప్పుడు నియమాలు ఇవి – మరియు **వాటి క్రమం చాలా ముఖ్యం**:

```
submit(task) {
  if (this.threads < this.core)        return this.spawn(task);    // 1
  if (this.q.length < this.queueCap)   return this.enqueue(task);  // 2
  if (this.threads < this.max)         return this.spawn(task);    // 3
  return this.reject(task);                                        // 4
}
```

ఈ క్రమం Java యొక్క `ThreadPoolExecutor` lo అచ్చు ఇలాగే ఉంది, మరియు చాలా మంది దీన్ని ఇలా చదువుతారు:

> "core threads నిండాక, queue నిండేవరకు పెడతాం, తర్వాత max వరకు threads పెంచుతాం."

ఆ వాక్యం సరైనదే. కానీ దానిలో ఒక **ఉచ్చు** ఉంది, మరియు అది ఒక్క పదంలో దాగి ఉంది: **"queue నిండేవరకు"**.

4. మొదటి విరుపు – 50 అడిగారు, 2 వచ్చాయి

చాలామంది pool ని ఇలా అమర్చుతారు: `core: 2, max: 50, queue: అపరిమితం`. ఆలోచన: "సాధారణంగా 2 threads చాలు, రద్దీలో 50 వరకు పెరుగుతుంది, మరియు queue కి పరిమితి పెట్టి పనులు పోగొట్టుకోకూడదు."

కొలుద్దాం. 500 పనులు, ప్రతి 10 ms కి ఒకటి, ఒక్కో పని 100 ms. అంటే **ఒకేసారి 10 threads** కావాలి.

```
500 tasks · ప్రతి 10 ms కి ఒకటి · ఒక్కో task 100 ms
(అంటే ఒకేసారి 10 threads కావాలి — demand = 100/10)

  అమరిక                         | నిజంగా వాడిన threads | queue గరిష్ఠం | p50    | p99    |
తిరస్కృతం
  -----------------------------+-------------------+----------+-------+--------+------
----
  core 2 · max 50 · queue అపరిమితం |                 2 |      400 | 10100ms | 19860ms
|        0
```

```
  core 2 · max 50 · queue 10   |                11 |        10 | 150ms |  280ms |
0
  core 10 · max 50 · queue అపరిమితం |                10 |         1 | 100ms |  100ms |
0
```

మొదటి వరుస – `MAX: 50` అని రాశారు, 2 THREADS నడిచాయి

అది ఒక bug కాదు. అది నియమాల యొక్క **సరైన** ఫలితం.

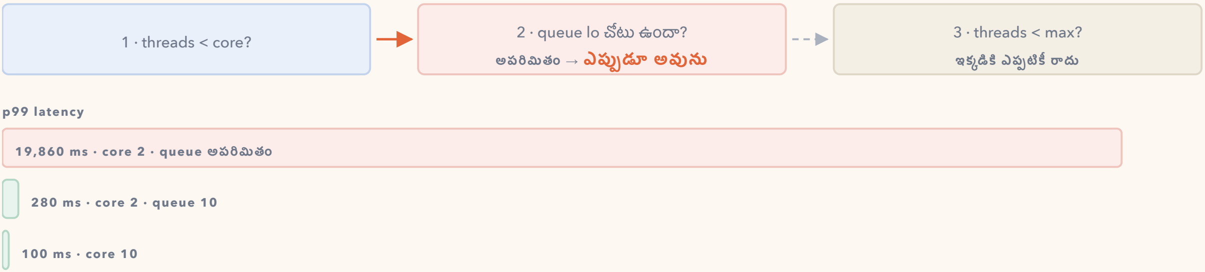
నియమం 2 చెప్తోంది: "queue lo చోటు ఉంటే queue lo పెట్టు." Queue **అపరిమితం**, కాబట్టి అందులో **ఎప్పుడూ** చోటు ఉంటుంది. కాబట్టి నియమం 2 ఎప్పుడూ గెలుస్తుంది, మరియు **నియమం 3 ఎప్పటికీ నడవదు**.

`max: 50` అనేది ఒక **చచ్చిన అమరిక**. అది ఏ ప్రభావమూ చూపని ఒక సంఖ్య.

పర్యవసానం: p99 **19,860 ms**. దాదాపు **20 సెకన్లు**, 100 ms పనికి.

మరియు గమనించండి – **తిరస్కృతం 0**. ఒక్క పని కూడా పోలేదు. మీ error dashboard **పచ్చగా** ఉంటుంది. Threads ఎప్పుడూ busy, కాబట్టి thread utilization కూడా **100%** – అది "ఆరోగ్యం" లా కనిపిస్తుంది.

అపరిమిత queue – నియమం 3 కి ఎప్పటికీ వంతు రాదు



Queue ని **చిన్నది** చేస్తే system **వేగం** అవుతుంది – 19,860 ms నుంచి 280 ms కి.

ఎందుకంటే **నిండిన queue ఒక్కటే** pool ని పెరగనిస్తుంది.

Queue అనేది ఒక దిండు కాదు – అది pool కి "ఇక చాలు" అని చెప్పే సంకేతం.

---

5. Step – queue ని చిన్నది చేయడం

పరిష్కారం ప్రతి-సహజమైనది: **queue ని పరిమితం చేయండి.**

రెండో వరుసలో `queue: 10` – threads **11** కి పెరిగాయి, p99 **19,860 ms నుంచి 280 ms** కి పడింది. **71 రెట్లు మెరుగు**, మరియు ఇప్పటికీ ఒక్క పని కూడా తిరస్కరించబడలేదు.

QUEUE యొక్క అసలు పని

Queue ఒక **నిల్వ గది** కాదు. అది pool కి **"నా సామర్థ్యం దాటింది, పెరుగు"** అని చెప్పే ఏకైక మార్గం.

అపరిమిత queue ఆ సంకేతాన్ని **శాశ్వతంగా మూసేస్తుంది**. Pool ఎప్పటికీ ఒత్తిడిని చూడదు, కాబట్టి ఎప్పటికీ పెరగదు.

Queue ఒక **చిన్న ఊపిరి తీసుకునే చోటు** గా ఉండాలి – ఉప్పెనల్ని మృదువు చేయడానికి, పనిని దాచిపెట్టడానికి కాదు.

మరియు మూడో వరుస ఇంకొక విషయం చెప్తోంది: `core: 10` పెడితే (queue అపరిమితమైనా) p99 **100 ms** – అంటే **ఎదురుచూపు అసలు లేదు**. అది ఈ మూడింటిలో ఉత్తమమైనది.

కాబట్టి అసలు సలహా ఏమిటి?

మీ సాధారణ భారానికి సరిపడా `core` పెట్టండి. `max` అనేది ఉప్పెనలకి మాత్రమే, మరియు అది పని చేయాలంటే `queueCap` **తప్పక పరిమితం** కావాలి.

`core` చిన్నది పెట్టి `max` పెద్దది పెట్టడం – అపరిమిత queue తో – అనేది **మూడు తప్పులు ఒకేసారి**: threads పెరగవు, latency పెరుగుతుంది, మరియు memory పెరుగుతుంది.

ఈ అమరిక తప్పు ఎప్పుడూ జరగకూడదు కాబట్టి, దాన్ని **constructor lo ఆపేద్దాం** – మౌనంగా తప్పు ప్రవర్తన కంటే బిగ్గరగా ఒక error మేలు:

```
constructor(sim, { core = 2, max = core, queueCap = Infinity, ... } = {}) {
  if (max < core) throw new Error('max కి core కంటే తక్కువ ఉండకూడదు');
  if (core < 1)   throw new Error('core కనీసం 1 ఉండాలి');
  // queue అపరిమితం అయితే max ఎప్పటికీ చేరదు — ఇది ఒక అమరిక తప్పు
  if (queueCap === Infinity && max > core)
    throw new Error('queue అపరిమితం అయితే max ఎప్పటికీ వాడబడదు — ' +
                    'queueCap పెట్టండి, లేదా max = core');
}
```

```
అమరికని constructor lo పరీక్షించడం:

  ✗ core 2 · max 50 · queue అపరిమితం    → queue అపరిమితం అయితే max ఎప్పటికీ వాడబడదు —
                                          queueCap పెట్టండి, లేదా max = core
  ✓ core 2 · max 50 · queue 100         → అంగీకరించబడింది
  ✓ core 10 · max 10 · queue అపరిమితం   → అంగీకరించబడింది
  ✗ core 5 · max 2                      → max కి core కంటే తక్కువ ఉండకూడదు
  ✗ core 0                              → core కనీసం 1 ఉండాలి
```

మూడో వరుసని గమనించండి – `core 10 · max 10 · queue అపరిమితం` **అంగీకరించబడింది**. ఎందుకంటే `max === core` అయితే అపరిమిత queue lo తప్పేమీ లేదు; pool ఎలాగూ పెరగదని మీరు **స్పష్టంగా** చెప్పారు.

మొదటి విరుపు పూర్తయింది. ఇప్పుడు pool సరిగ్గా పెరుగుతుంది. తర్వాతి విరుపు pool పరిమాణం గురించి కాదు – అది **ఎవరు pool ని అడుగుతున్నారు** అనే దాని గురించి.

Part 3 – రెండో విరుపు: పని తనకే పని ఇస్తే

6. Step – ఒక పని లోపల ఇంకొక పని

ఈ code ని చూడండి. ఇది ఒక వెబ్ request ని నిర్వహిస్తోంది:

```
pool.submit({
  run: done => {
    const user = fetchUser(id);
    // చిత్రాన్ని సిద్ధం చేయడం ఒక వేరే పని — దాన్ని pool కి ఇద్దాం
    pool.submit({ run: imgDone => { resizeAvatar(user); imgDone(); } });
    // ...మరియు అది అయ్యేవరకు ఎదురుచూసి, తర్వాత జవాబు పంపుదాం
    waitForImage(() => { respond(user); done(); });
  }
});
```

ఇందులో ఏమీ తప్పు కనిపించదు. "ఒక పెద్ద పనిని చిన్న పనులుగా విడగొట్టి pool కి ఇవ్వడం" అనేది **మంచి పద్ధతి** గా నేర్పబడుతుంది. Parallel streams, `CompletableFuture`, fork-join – అన్నీ ఈ ఆకారంలోనే ఉంటాయి.

కానీ ఇక్కడ ఒక **లెక్క** ఉంది, మరియు ఎవరూ దాన్ని చేయరు.

ఆ లెక్క ఇది

మీ pool lo **N threads** ఉన్నాయి.

N పనులు ఒకేసారి వచ్చాయి. ప్రతి ఒక్కటీ ఒక thread తీసుకుంది. **ఇప్పుడు ఖాళీ threads: 0.**

ప్రతి ఒక్కటీ ఒక ఉప-పనిని submit చేసింది. ఆ N ఉప-పనులు **queue lo** ఉన్నాయి.

ఉప-పని నడవాలంటే ఒక thread కావాలి. Thread ఖాళీ కావాలంటే ఒక parent పూర్తవ్వాలి. Parent పూర్తవ్వాలంటే దాని ఉప-పని అవ్వాలి.

ఇది ఒక వలయం. అది ఎప్పటికీ తెగదు.

7. రెండో విరుపు – 7 బాగుంది, 8 శాశ్వతంగా ఆగింది

కొలుద్దాం. 8 threads, ప్రతి పనీ ఒక ఉప-పనిని submit చేసి దాని కోసం ఎదురుచూస్తుంది:

```
8 threads · 8 parent tasks
```

```
  ఉప-పని లేకుండా
    పూర్తయినవి: 8 · ఖాళీ threads: 8 · queue lo: 0
    కాలక్రమం: 0ms[ఖాళీ 0·queue 0·అయినవి 0] 500ms[ఖాళీ 8·queue 0·అయినవి 8] ...

  ప్రతిదీ ఒక ఉప-పని కోసం ఎదురుచూస్తే
    పూర్తయినవి: 0 · ఖాళీ threads: 0 · queue lo: 8
    కాలక్రమం: 0ms[ఖాళీ 0·queue 8·అయినవి 0] 500ms[ఖాళీ 0·queue 8·అయినవి 0]
               1000ms[ఖాళీ 0·queue 8·అయినవి 0] ... 3000ms[ఖాళీ 0·queue 8·అయినవి 0]
```

ఆ రెండో కాలక్రమాన్ని చూడండి. **అది ఏ మాత్రం మారలేదు.** 0 ms దగ్గర ఏమి ఉందో 3,000 ms దగ్గరా అదే. నేను simulation ని 100,000 ms వరకు నడిపాను – ఒక్క సంఖ్య కూడా కదల్లేదు.

ఇప్పుడు ముఖ్యమైన ప్రశ్న: **ఇది ఎన్ని పనుల దగ్గర మొదలవుతుంది?**

```
8 threads · ప్రతి పనీ ఒక ఉప-పని కోసం ఎదురుచూస్తుంది · అన్నీ ఒకే pool

  ఒకేసారి వచ్చిన పనులు | పూర్తయినవి | ఖాళీ threads | queue lo చిక్కుకున్నవి | స్థితి
  --------------------+----------+----------+---------------+--------
                    4 |        4 |        8 |             0 | బాగుంది
                    6 |        6 |        8 |             0 | బాగుంది
                    7 |        7 |        8 |             0 | బాగుంది
                    8 |        0 |        0 |             8 | ⚠ పూర్తిగా ఆగింది
                    9 |        0 |        0 |             9 | ⚠ పూర్తిగా ఆగింది
                   16 |        0 |        0 |            16 | ⚠ పూర్తిగా ఆగింది
```

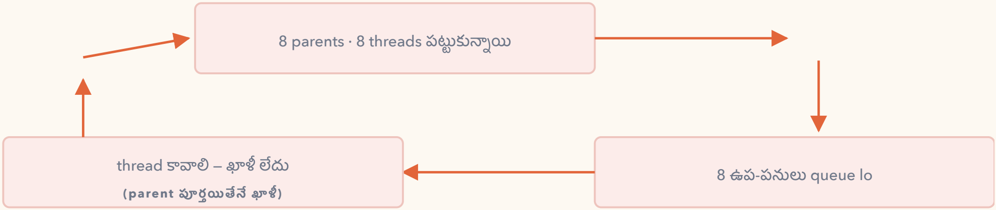
7 మరియు 8 మధ్య ఉన్న గీత

7 పనులు: **అన్నీ పూర్తి**. 8 పనులు: **ఏదీ పూర్తి కాదు, ఎప్పటికీ**.

కొంచెం నెమ్మది కాదు. 50% నెమ్మది కాదు. **సున్నా**, మరియు అది శాశ్వతం. Restart తప్ప వేరే దారి లేదు.

మరియు ఇది అత్యంత ప్రమాదకరమైన భాగం: **మీ code 7 దగ్గరా 8 దగ్గరా అచ్చు ఒకటే.** మీరు ఏమీ మార్చలేదు. కేవలం **traffic** మారింది.

ఒక వలయం – మరియు దాన్ని ఏ timeout కూడా తెంచదు



ఇది ఒక **నెమ్మదితనం కాదు** – ఇది ఒక పూర్తి, శాశ్వతమైన ఆగిపోవడం.

Timeout పెట్టినా ఉపయోగం లేదు: గడువు దాటిన పని **మళ్ళీ అదే వలయంలోకే** వెళ్తుంది.

Pool ని పెద్దది చేసినా ఉపయోగం లేదు: రేపు ఇంకొక్క పని ఎక్కువ వస్తే మళ్ళీ అదే.

ఏకైక పరిష్కారం – **వలయాన్నే తొలగించడం**.

8. Step – bulkhead · వేరే పనికి వేరే pool

వలయాన్ని తెంచడానికి **రెండు** దారులు ఉన్నాయి, మరియు రెండూ తెలుసుకోవాలి.

దారి ఒకటి – అసలు ఎదురుచూడకపోవడం (ఉత్తమమైనది)

ఉప-పని కోసం thread ని పట్టుకుని ఎదురుచూడటం మానేసి, **దాని తర్వాత ఏమి చెయ్యాలో చెప్పి thread ని వదిలేయడం**:

```
// ఎదురుచూడటం — thread ఇంకా పట్టుకునే ఉంది
pool.submit({ run: done => {
  const sub = pool.submit(childTask);
  sub.await();                        // ← ఇక్కడే deadlock
  done();
}});

// ఎదురుచూడకపోవడం — thread వెంటనే విడుదల
pool.submit({ run: done => {
  done();                             // ← నా పని అయిపోయింది
  pool.submit({ run: d2 => { finishUp(); d2(); } });
}});
```

ఇది **నిజమైన** పరిష్కారం. `CompletableFuture.thenApply`, `Promise.then`, `async/await` – ఈ అన్నిటి ఉనికికి కారణం ఇదే: **ఎదురుచూపుని thread నుంచి విడదీయడం**.

కానీ ఇది ఎప్పుడూ సాధ్యం కాదు. మీరు వాడుతున్న library `blocking` కావచ్చు. అప్పుడు:

దారి రెండు – bulkhead

ఉప-పనులని **వేరే pool** కి పంపడం. అప్పుడు parents ఎంతమంది ఎదురుచూసినా, ఉప-పనులకి **సొంత threads** ఉంటాయి:

```
const main = new ThreadPool(sim, { core: 8, max: 8, queueCap: 100 });
const kids = new ThreadPool(sim, { core: 4, max: 4, queueCap: 100 });

main.submit({ run: done =>
  kids.submit({ run: kdone => { work(); kdone(); done(); } })   // ← వేరే pool
});
```

```
పరిష్కారం: ఉప-పనులకి వేరే pool (bulkhead)

  ఒకేసారి వచ్చిన పనులు | పూర్తయినవి | స్థితి
  --------------------+----------+--------
                    8 |        8 | బాగుంది
                   16 |       16 | బాగుంది
                   64 |       64 | బాగుంది
```

64 పనులు, 8 threads మాత్రమే – అయినా అన్నీ పూర్తయ్యాయి. ఎందుకంటే వలయం తెగింది: ఉప-పని ఇప్పుడు parent కోసం ఎదురుచూసే pool నుంచి thread అడగడం లేదు.

BULKHEAD – ఈ పేరు ఎక్కడి నుంచి వచ్చింది, మరియు ఎందుకు సరైనది

ఓడలో **bulkhead** అంటే నీరు రాకుండా అడ్డుగా ఉండే గోడ. ఒక గదిలో రంధ్రం పడితే ఆ గది మాత్రమే మునుగుతుంది, ఓడ మునగదు.

ఇక్కడా అదే: **ఒక రకమైన పని ఇంకొక రకాన్ని ఆకలితో చంపకూడదు.**

మరియు ఇది deadlock కి మాత్రమే కాదు – **నెమ్మదైన పనులకి కూడా**. ఒక pool lo వేగమైన పనులు, నెమ్మదైనవి కలిపితే, నెమ్మదైనవి అన్ని threads ని పట్టుకుని వేగమైన వాటిని ఆపేస్తాయి. వేరు చేస్తే ఒకటి ఇంకొకదాన్ని ముట్టుకోదు.

INTERVIEW LO ఈ విరుపు ఎందుకు ముఖ్యమైనది

Thread pool deadlock గురించి చాలామంది "ఉంటుంది" అని చెప్తారు. మీరు **"8 threads తో 7 పనులు బాగా నడుస్తాయి, 8వది సున్నా throughput ఇస్తుంది, శాశ్వతంగా – మరియు code ఒక్క అక్షరం మారలేదు"** అని చెప్పగలిగితే, అది పూర్తిగా వేరే స్థాయి.

ఎందుకంటే అది చెప్పేది ఏమిటంటే – **ఈ bug ని test lo పట్టుకోలేరు**. Test ఎప్పుడూ తక్కువ concurrency తో నడుస్తుంది. ఇది **అత్యధిక రద్దీలో మాత్రమే** కనిపిస్తుంది, అంటే మీ అత్యంత చెత్త క్షణంలో.

Part 4 – మూడో విరుపు: నిండినప్పుడు

9. Step – నిండితే ఏమి చెయ్యాలి?

Pool నిండింది. Queue నిండింది. కొత్త పని వచ్చింది. ఇప్పుడు **తప్పనిసరిగా** ఏదో ఒకటి చెయ్యాలి, మరియు ప్రతి ఎంపికకీ ఒక ధర ఉంది.

మూడు దారులు:

దారి	ఏమి చేస్తుంది	ఎవరికి నొప్పి
తిరస్కరించు	పనిని వదిలేసి caller కి "లేదు" అని చెప్పు	caller – కానీ అతనికి **వెంటనే తెలుస్తుంది**
పాతదాన్ని వదిలేయ్	queue lo అతి పాత పనిని తీసేసి కొత్తదాన్ని పెట్టు	ఆ పాత పనిని పంపినవాడు – **మౌనంగా**
caller చేస్తాడు	submit చేసినవాడే ఆ పనిని నడుపుతాడు	caller – అతను **ఆగిపోతాడు**

మరియు నాలుగో "దారి", అందరూ default గా వాడేది: **అపరిమిత queue** – అంటే ఎప్పుడూ నిండదు, కాబట్టి ఈ నిర్ణయమే తీసుకోనవసరం లేదు.

అది ఒక దారి కాదు. అది **నిర్ణయాన్ని వాయిదా వేయడం** – §4 lo చూసినట్టు.

10. మూడో విరుపు – మూడు దారులు, మూడింటికీ ఖరీదు

కొలుద్దాం. 4 threads, ఒక్కో పని 50 ms – అంటే సామర్థ్యం **80/సెకను**. Producer **200/సెకను** ఇస్తున్నాడు, అంటే **2.5 రెట్లు ఎక్కువ**. 2,000 పనులు.

```
4 threads · ఒక్కో పని 50 ms (సామర్థ్యం: 80/సె) · producer 200/సె ఇస్తున్నాడు
2,000 పనులు

  విధానం                | పూర్తయినవి | పోయినవి | queue గరిష్ఠం | p99      | producer ఆగిన
సమయం
  ---------------------+----------+---------+----------+---------+-----------------
  queue అపరిమితం        |    2000 |      0 |     1200 | 14900ms |                 –
  నిండితే తిరస్కరణ      |     850 |   1150 |       50 |   690ms |                 –
  నిండితే caller చేస్తాడు |    2000 |      0 |       50 |   680ms | 12.5 s (మొత్తంలో 59%)
```

మూడు వరుసలూ చదవండి. **ఏ ఒక్కటీ ఉచితం కాదు.**

అపరిమిత QUEUE – "అంతా విజయవంతం", 14.9 సెకన్ల LATENCY తో

2,000/2,000 పూర్తయ్యాయి, సున్నా పోయాయి. కానీ queue **1,200** కి పెరిగింది మరియు p99 **14,900 ms**.

Queue lo ఉన్న 1,200 పనులు **memory**. Producer ఇంకా వేగంగా ఇస్తే అది 12,000 అవుతుంది, తర్వాత 1,20,000 – మరియు **process చచ్చిపోతుంది**. §4 lo ఇదే వ్యాధి, ఇక్కడ ఇంకా స్పష్టంగా.

మరియు ఆ 14.9 సెకన్ల తర్వాత పూర్తయిన పనికి **విలువ ఉందా?** User ఎప్పుడో వెళ్ళిపోయాడు.

తిరస్కరణ – వేగం, కానీ 57% పని పోయింది

p99 **690 ms**, queue 50 దగ్గరే ఆగింది, memory సురక్షితం. కానీ **2,000 lo 1,150 తిరస్కరించబడ్డాయి** – **57.5%**.

ఇది చెడ్డది కాదు ఎప్పుడంటే caller దాన్ని నిర్వహించగలిగితే – మళ్ళీ ప్రయత్నించడం, లేదా user కి "ఇప్పుడు రద్దీగా ఉంది" అని చెప్పడం. **తక్షణ, స్పష్టమైన వైఫల్యం** ఒక నిజాయితీ.

కానీ ఆ పని **పోగొట్టుకోకూడనిది** అయితే – ఒక చెల్లింపు, ఒక order – అప్పుడు ఇది ఒక విపత్తు.

CALLER చేస్తాడు – అన్నీ పూర్తి, QUEUE పరిమితం, P99 తక్కువ... ఒక షరతుతో

2,000/2,000 పూర్తి, సున్నా పోయాయి, queue 50, p99 **680 ms** – తిరస్కరణ కంటే కూడా కొంచెం మెరుగు.

ఇది **backpressure**: producer వేగంగా ఇవ్వలేడు, ఎందుకంటే ఇవ్వడానికి ప్రయత్నిస్తే అతనే ఆ పని చేయాల్సి వస్తుంది. వ్యవస్థ **తనని తానే** సమతుల్యం చేసుకుంటుంది.

కానీ చివరి నిలువు వరుస చూడండి: **producer మొత్తం సమయంలో 59% ఆగిపోయాడు.**

ఆ 59% ఒక అమాయకమైన సంఖ్య కాదు. **Producer ఎవరు?**

CALLER-RUNS యొక్క దాగిన ప్రమాదం

మీ producer ఒక HTTP **acceptor thread** అయితే – అంటే కొత్త connections ని స్వీకరించే thread – అప్పుడు caller-runs అతన్ని 50 ms పాటు ఆపేస్తుంది.

ఆ 50 ms lo మీ server **ఏ కొత్త connection నూ స్వీకరించదు**. Health check తో సహా.

Load balancer మీ health check కి జవాబు రాకపోవడం చూస్తుంది. అది మీ server ని **తీసేస్తుంది**. మిగిలిన servers మీద భారం పెరుగుతుంది. అవీ ఇదే చేస్తాయి.

Caller-runs ఒక అద్భుతమైన backpressure సాధనం – కానీ caller ఎవరో మీకు తెలిసినప్పుడే.

---

11. Step – నిర్ణయాన్ని బయటికి తీయడం

DD 25 §11 lo catch-up విధానం లాగే, ఇక్కడ కూడా **సరైన సమాధానం pool కి తెలియదు**. అది వాడే సందర్భాన్ని బట్టి ఉంటుంది.

కాబట్టి దాన్ని ఒక **విలువ** చేద్దాం:

```
const OnFull = {
  reject:     () => ({ kind: 'reject' }),
  discardOld: () => ({ kind: 'discardOld' }),   // queue lo అతి పాతదాన్ని వదిలేయ్
  callerRuns: () => ({ kind: 'callerRuns' }),   // ఇచ్చినవాడే చెయ్యాలి
};
```

మరియు pool కేవలం దాన్ని పిలుస్తుంది:

```
#full(task) {
  const d = this.#onFull();
  if (d.kind === 'reject') {
    this.#stats.rejected++;
    return { accepted: false, reason: 'FULL' };
  }
  if (d.kind === 'discardOld') {
    const dropped = this.#q.shift();
    this.#stats.discarded++;
    dropped?.onDiscard?.();              // ← మౌనంగా పోకూడదు
    return this.#enqueue(task);
  }
  this.#stats.callerRan++;
  this.#sim.after(task.ms, () => {
    this.#stats.completed++; task.onDone?.(this.#sim.now);
  });
  return { accepted: true, where: 'caller', blockFor: task.ms };
}
```

రెండు వివరాలు ముఖ్యమైనవి:

`dropped?.onDiscard?.()` – ఒక పనిని వదిలేస్తే, దాన్ని పంపినవాడికి **చెప్పాలి**. లేకపోతే అది ఒక మౌన వైఫల్యం, మరియు మౌన వైఫల్యాలే అత్యంత ఖరీదైనవి.

`return { accepted: true, where: 'caller', blockFor: task.ms }` – caller-runs lo pool **ఎంతసేపు ఆగాల్సి వచ్చిందో caller కి తిరిగి చెప్తుంది**. అలా caller ఆ సంఖ్యని కొలవగలడు, మరియు "నేను 59% సమయం ఆగిపోతున్నాను" అని **తెలుసుకోగలడు** – §10 lo మనం కొలిచిన సంఖ్య అదే.

> **ఒక DESIGN నియమం – మూడు DOCS LO మూడోసారి**
>
> DD 24 lo pool పరిమాణం **DBA ది**. DD 25 lo catch-up విధానం **job యజమాని ది**. ఇక్కడ నిండినప్పటి విధానం **caller ది**.
>
> మూడూ ఒకే ఆకారం: **వ్యవస్థకి తెలియని నిర్ణయాన్ని వ్యవస్థ తీసుకోకూడదు. దాన్ని మోసుకెళ్ళాలి.**

Part 5 – పూర్తి system

12. Step – ఎన్ని threads?

ఇది thread pool గురించి అడిగే **అత్యంత సాధారణమైన** ప్రశ్న, మరియు దీనికి ఒక సూత్రం ఉంది:

> **threads = cores × (1 + ఎదురుచూపు ÷ CPU పని)**

ఈ సూత్రాన్ని అందరూ ఉదహరిస్తారు. దాన్ని **కొలిచి చూద్దాం** – నిజంగా పని చేస్తుందా?

4 cores. ఒక్కో పనిలో రెండు భాగాలు: CPU మీద నిజమైన లెక్క, మరియు ఎదురుచూపు (network, disk).

```
CPU పని ఎక్కువ · CPU 10 ms · ఎదురుచూపు 0 ms · cores 4
సూత్రం చెప్పేది: 4 × (1 + 0/10) = 4

  threads | throughput          threads | throughput
  --------+-----------          --------+-----------
        2 |     200/s                16 |     400/s
        4 |     400/s                32 |     400/s
        8 |     400/s                64 |     400/s
  → అత్యుత్తమం: 4 threads · సూత్రం చెప్పినది 4
```

CPU-బౌండ్ పనికి **4 threads దాటి ఏమీ మారలేదు** – 8, 16, 64 అన్నీ సరిగ్గా 400/s. ఎందుకంటే cores 4 మాత్రమే; మిగతా threads కేవలం ఎదురుచూస్తాయి.

```
సగం సగం · CPU 10 ms · ఎదురుచూపు 10 ms · cores 4
సూత్రం చెప్పేది: 4 × (1 + 10/10) = 8

  threads | throughput
  --------+-----------
        2 |     100/s
        4 |     200/s
        8 |     397/s
       16 |     397/s
  → అత్యుత్తమం: 8 threads · సూత్రం చెప్పినది 8
```

```
ఎదురుచూపు ఎక్కువ (DB call) · CPU 2 ms · ఎదురుచూపు 20 ms · cores 4
సూత్రం చెప్పేది: 4 × (1 + 20/2) = 44

  threads | throughput
  --------+-----------
        2 |      91/s
        4 |     182/s
        8 |     363/s
       16 |     716/s
```

```
      32 |    1402/s
      40 |    1724/s
      44 |    1875/s
      48 |    1875/s
      64 |    1875/s
  → అత్యుత్తమం: 44 threads · సూత్రం చెప్పినది 44
```

సూత్రం మూడు సార్లూ సరిగ్గా చెప్పింది

4 → 4. 8 → 8. 44 → **44** (43 దగ్గర 1,724/s, 44 దగ్గర 1,875/s, ఆ తర్వాత చదునుగా).

మరియు ఇక్కడ అసలు పాఠం ఆ సంఖ్యలు కాదు – **వాటి మధ్య ఉన్న దూరం**:

ఒకే machine, ఒకే cores. CPU పనికి **4 threads**. Database పనికి **44**. **11 రెట్లు తేడా.**

కాబట్టి "మా server కి 4 cores, కాబట్టి 8 threads పెడదాం" అనే సలహా **పని రకం తెలియకుండా అర్థరహితం**.

కానీ – ఆ 44 ని గుడ్డిగా వాడకండి

§12 lo మనం లెక్కించిన 44 అనేది **మన CPU** నిండకుండా ఉండే గరిష్ఠం. కానీ ఆ 44 threads ఒక database ని పిలుస్తుంటే, అప్పుడు DD 24 §12 వర్తిస్తుంది: **database ఎన్ని ఏకకాల queries భరిస్తుందో అదే నిజమైన పరిమితి**, మరియు అది తరచుగా 44 కంటే చాలా తక్కువ.

అంటే మీ thread pool పరిమాణం **రెండింటిలో చిన్నది**:

- `cores × (1 + ఎదురుచూపు/CPU)` – మీ CPU నిండకుండా
- downstream యొక్క సామర్థ్యం – దాన్ని ముంచకుండా

ఈ రెండు docs కలిపి చదివితే వచ్చే ముగింపు ఇది, మరియు interview lo ఇది చెప్తే గుర్తుండిపోతారు.

---

13. మొత్తం code · 8.5 లక్షల tasks · mutation testing

```
const OnFull = {
  reject:     () => ({ kind: 'reject' }),
  discardOld: () => ({ kind: 'discardOld' }),
  callerRuns: () => ({ kind: 'callerRuns' }),
};

class ThreadPool {
  #sim; #core; #max; #queueCap; #onFull; #keepAliveMs;
  #q = []; #threads = 0; #busy = 0; #closed = false; #drained = null;
  #stats = { submitted: 0, started: 0, completed: 0, rejected: 0, discarded: 0,
             callerRan: 0, spawned: 0, reaped: 0, peakThreads: 0, peakQ: 0 };

  constructor(sim, { core = 2, max = core, queueCap = Infinity,
                     onFull = OnFull.reject, keepAliveMs = Infinity } = {}) {
    if (max < core) throw new Error('max కి core కంటే తక్కువ ఉండకూడదు');
    if (core < 1)   throw new Error('core కనీసం 1 ఉండాలి');
    if (queueCap === Infinity && max > core)
      throw new Error('queue అపరిమితం అయితే max ఎప్పటికీ వాడబడదు – ' +
```

```
                    'queueCap పెట్టండి, లేదా max = core');
    this.#sim = sim; this.#core = core; this.#max = max;
    this.#queueCap = queueCap; this.#onFull = onFull;
    this.#keepAliveMs = keepAliveMs;
  }
```

§3 – నాలుగు నియమాలు, ఆ క్రమంలోనే:

```
submit(task) {
  if (this.#closed) return { accepted: false, reason: 'CLOSED' };
  this.#stats.submitted++;
  if (this.#threads < this.#core)      return this.#spawn(task);     // 1
  if (this.#q.length < this.#queueCap) return this.#enqueue(task);   // 2
  if (this.#threads < this.#max)       return this.#spawn(task);     // 3
  return this.#full(task);                                           // 4
}
```

§11 – నిండితే, caller చెప్పిన విధానం:

```
#full(task) {
  const d = this.#onFull();
  if (d.kind === 'reject') {
    this.#stats.rejected++;
    return { accepted: false, reason: 'FULL' };
  }
  if (d.kind === 'discardOld') {
    const dropped = this.#q.shift();
    this.#stats.discarded++;
    dropped?.onDiscard?.();                    // మౌనంగా పోకూడదు
    return this.#enqueue(task);
  }
  this.#stats.callerRan++;
  this.#sim.after(task.ms, () => {
    this.#stats.completed++; task.onDone?.(this.#sim.now);
  });
  return { accepted: true, where: 'caller', blockFor: task.ms };
}
```

Queue lo పెట్టడం, thread తయారు చేయడం, మరియు idle threads ని తీసేయడం:

```
#enqueue(task) {
  this.#q.push(task);
  this.#stats.peakQ = Math.max(this.#stats.peakQ, this.#q.length);
  this.#pump();
  return { accepted: true, where: 'queue' };
}

#spawn(task) {
  this.#threads++;
  this.#stats.spawned++;
  this.#stats.peakThreads = Math.max(this.#stats.peakThreads, this.#threads);
  this.#start(task);
  return { accepted: true, where: 'thread' };
```

```
  }

  #pump() {
    while (this.#busy < this.#threads && this.#q.length)
      this.#start(this.#q.shift());
  }

  // core కి మించిన threads ఖాళీగా ఉంటే కొంతసేపటికి వాటిని తీసేయ్
  #maybeReap() {
    if (this.#threads <= this.#core) return;      // core threads ఎప్పుడూ ఉంటాయి
    if (this.#keepAliveMs === Infinity) return;
    this.#sim.after(this.#keepAliveMs, () => {
      if (this.#busy < this.#threads && this.#threads > this.#core) {
        this.#threads--; this.#stats.reaped++;
      }
    });
  }
```

నడపడం – మరియు ఒక సూక్ష్మమైన, కీలకమైన వివరం:

```
  #start(task) {
    this.#busy++;
    this.#stats.started++;
    // నిజమైన pool lo submit() వెంటనే తిరిగి వస్తుంది — పని తర్వాత నడుస్తుంది
    this.#sim.after(0, () => this.#sim.after(task.ms, () => {
      this.#busy--;
      this.#stats.completed++;
      task.onDone?.(this.#sim.now);
      if (this.#q.length) return this.#start(this.#q.shift());
      this.#maybeReap();
      if (this.#closed && this.#busy === 0 && !this.#q.length) this.#drained?.();
    }));
  }
```

ఆ `AFTER(0, ...)` – అది లేకపోతే నా §7 కొలత తప్పుగా వచ్చింది

నా మొదటి simulation lo పనిని నేరుగా `#pump()` లోపలే పిలిచాను. దాంతో ఒక parent **వెంటనే** నడిచి, తన ఉప-పనిని submit చేసి, ఆ ఉప-పని **తర్వాతి parent submit అవ్వకముందే** ఒక thread తీసుకుంది.

ఫలితం: 8 parents lo **4 మాత్రమే** queue lo చిక్కుకున్నాయి, మరియు **deadlock రాలేదు** – 16 పనులు పూర్తయ్యాయి.

కానీ నిజమైన thread pool lo `execute()` **వెంటనే తిరిగి వస్తుంది**; పని ఒక వేరే thread మీద తర్వాత నడుస్తుంది. అంటే 8 parents అన్నీ threads తీసుకున్న **తర్వాతే** అవి ఉప-పనులని submit చేస్తాయి.

`after(0, ...)` చేర్చాక deadlock **వెంటనే** కనిపించింది. **నా simulation నిజం కంటే దయగా ఉంది**, మరియు అది ఒక నిజమైన, ఘోరమైన bug ని దాచేసింది.

మూసివేత – మిగిలిన పనిని *తిరిగి* ఇవ్వడం:

```
  // మిగిలిన పనిని తిరిగి ఇస్తుంది — మౌనంగా పారేయకూడదు
  shutdownNow() {
```

```
    this.#closed = true;
    return this.#q.splice(0);
  }
  shutdown(onDone) {
    this.#closed = true;
    if (this.#busy === 0 && !this.#q.length) return onDone?.();
    this.#drained = onDone;
  }
```

ఎందుకు `SHUTDOWNNOW` పనిని తిరిగి ఇస్తుంది

చాలా implementations queue ని ఖాళీ చేసి ఊరుకుంటాయి. అప్పుడు ఆ పని **మౌనంగా అదృశ్యమవుతుంది**.

తిరిగి ఇస్తే, పిలిచినవాడు ఏమి చెయ్యాలో నిర్ణయించగలడు – ఇంకొక pool కి ఇవ్వడం, disk lo రాయడం, లేదా కనీసం **"47 పనులు పోయాయి" అని log చేయడం**.

Java యొక్క `ExecutorService.shutdownNow()` కూడా సరిగ్గా ఇదే చేస్తుంది, మరియు దానికి కారణం ఇదే.

లెక్కలు:

```
  stats() {
    return { ...this.#stats, threads: this.#threads, busy: this.#busy,
             queued: this.#q.length, core: this.#core, max: this.#max };
  }
}
```

ఐదు నియమాలు

#	నియమం	విరిగితే అర్థం
1	`threads <= max`	pool పరిమితి దాటి పెరుగుతోంది
2	`busy <= threads`	లేని thread మీద పని నడుస్తోంది
3	`queued <= queueCap`	queue పరిమితి పని చేయడం లేదు
4	ఒక పని రెండుసార్లు నడవకూడదు	అప్పగింత విరిగింది
5	**ప్రతి పనికీ ఒక గమ్యం** – పూర్తి, తిరస్కరణ, వదిలివేత, లేదా వెనక్కి	ఒక పని మౌనంగా పోయింది

నియమం 5 అత్యంత ముఖ్యమైనది, మరియు అది ఒక **లెక్క**:

```
const accounted = completed + rejected + discarded
                + leftover.length + queued + busy;
if (accounted !== submitted)
  bad.push(`లెక్క తప్పింది: ఇచ్చినవి ${submitted} ≠ ${accounted}`);
```

1,500 ప్రయోగాలు – ప్రతిదానిలో యాదృచ్ఛిక `core`, `max`, `queueCap`, విధానం, `keepAlive`, మరియు మధ్యలో హఠాత్తుగా `shutdownNow()`:

```
1,500 యాదృచ్ఛిక ప్రయోగాలు · 8,47,653 tasks ఇచ్చాం
  నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:
  submitted   8,47,653
  completed   6,32,496
  rejected    1,05,876
  discarded     97,812
  callerRan     96,509
  spawned        9,642
  reaped         4,445
  started     5,35,987
```

Test విఫలం కాగలదా?

DD 24 మరియు DD 25 lo లాగే – **"0 ఉల్లంఘనలు" అంటే test పనికిరానిదని కూడా అర్థం కావచ్చు.** నాలుగు పంక్తులు ఒక్కొక్కటిగా తీసేసి చూశాను:

```
  మార్పు లేని code                        →    0/1500 విఫలం

  max పరీక్ష తీసేస్తే (నియమం 3)          →   385/400 విఫలం
     ఉదా: 400ms: threads=6 > max=3
  queue పరిమితి పరీక్ష తీసేస్తే            →   394/400 విఫలం
     ఉదా: 400ms: queue=14 > పరిమితి=11
  shutdownNow మిగిలినవి తిరిగి ఇవ్వకపోతే →   325/400 విఫలం
     ఉదా: లెక్క తప్పింది: ఇచ్చినవి 757 ≠ 746
  busy లెక్క తగ్గించకపోతే                 →   400/400 విఫలం
     ఉదా: 120ms: busy=4 > threads=3
```

నాలుగూ పట్టుబడ్డాయి. DD 25 lo ఒకటి తప్పించుకుంది (backoff – ఒక భారపు లక్షణం); ఇక్కడ నాలుగూ సరి-తప్పు లక్షణాలు, కాబట్టి నాలుగూ నియమాలతో పట్టుబడ్డాయి.

మూడో దాన్ని గమనించండి – `shutdownNow` మిగిలిన పనిని తిరిగి ఇవ్వకపోతే **325/400 విఫలం**, మరియు సందేశం `ఇచ్చినవి 757 ≠ 746`. అంటే **11 పనులు అదృశ్యమయ్యాయి**. ఒక్క error లేదు, ఒక్క exception లేదు – అవి కేవలం **లేవు**. నియమం 5 లేకపోతే ఇది ఎప్పటికీ కనిపించేది కాదు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ	ఏమి జోడించాం	ఏమి బాగుపడింది
§3	core/max/queue నియమాలు	pool ఉంది
§5	`queueCap` పరిమితం	p99 **19,860 ms → 280 ms**
§5	constructor lo అమరిక పరీక్ష	చచ్చిన `max` ఇక మౌనంగా ఉండదు
§8	వేరే pool (bulkhead)	8 పనుల దగ్గర deadlock → **64 పనులు సరే**
§11	`OnFull` విలువలు	నిండినప్పటి నిర్ణయం caller దే
§11	`onDiscard`, `blockFor`	వదిలినవి, ఆగిన సమయం రెండూ **కనిపిస్తాయి**

ఎక్కడ	ఏమి జోడించాం	ఏమి బాగుపడింది
§13	`shutdownNow` తిరిగి ఇవ్వడం	మూసివేతలో పని మాయం కాదు

Part 6 — Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెయ్యాలి
0–4	Pool ఒక limiter. **10,000 threads, 4 cores** ఉదాహరణ చెప్పండి
4–8	§2 ప్రశ్నలు. ముఖ్యంగా **"ఒక పని లోపల ఇంకొక పని submit చేస్తారా?"**
8–14	నాలుగు నియమాలు రాయండి. క్రమం ముఖ్యమని నొక్కి చెప్పండి
14–21	**మొదటి విరుపు.** "max 50 రాశారు, 2 వచ్చాయి" – నియమం 2 ఎప్పుడూ గెలుస్తుందని చూపించండి
21–30	**రెండో విరుపు.** Board మీద ఆ వలయాన్ని గీయండి. **7 vs 8** – ఇది గుర్తుండిపోతుంది
30–36	**మూడో విరుపు.** మూడు దారులు, మూడింటికీ ఖరీదు. caller-runs యొక్క acceptor thread ప్రమాదం చెప్పండి
36–41	**ఎన్ని threads?** సూత్రం, మరియు **4 vs 44**
41–45	ఐదు నియమాలు, ముఖ్యంగా **"ప్రతి పనికీ ఒక గమ్యం"**

ఏమి తప్పక చెప్పాలి

1. **అపరిమిత queue `max` ని చచ్చిన అమరికగా మారుస్తుంది.** Queue నిండటమే pool పెరగడానికి ఏకైక సంకేతం.
2. **ఒక పని ఇంకొక పనిని అడగగలదు – అదే thread pool ని connection pool నుంచి వేరు చేస్తుంది.** 7 బాగుంది, 8 శాశ్వతంగా ఆగుతుంది.
3. **నిండినప్పుడు ఉచిత ఎంపిక లేదు.** తిరస్కరణ పనిని పోగొడుతుంది, caller-runs producer ని ఆపుతుంది, అపరిమిత queue memory ని తింటుంది.
4. **Thread సంఖ్య పని రకం మీద ఆధారపడుతుంది** – ఒకే machine మీద 4 లేదా 44.
5. **Shutdown lo మిగిలిన పని తిరిగి ఇవ్వాలి**, పారేయకూడదు.

ఏమి వదిలేయాలి

- Lock-free queue అమలు వివరాలు – ఒక వాక్యం చాలు
- Thread priority – OS దాన్ని దాదాపు పట్టించుకోదు
- ThreadLocal శుభ్రత – అడిగితే మాత్రమే (కానీ అది ఒక నిజమైన leak మూలం)
- Work stealing – §16 lo ఒక వాక్యంలో

15. నోటితో చెప్పాల్సిన English script

> "A thread pool looks like a cache for threads, but it's really a **limiter**. You can create ten thousand threads; you have four cores. Those extra threads don't add throughput – they add ten gigabytes of stacks and enough context switching that the OS spends more time switching than working. The pool's job is to keep your machine safe from your own traffic."

> "There's one thing that makes this different from a connection pool, and it's the source of the worst bug here: **a query can't ask for another query, but a task can submit another task.** I'll come back to that."

> "The submit path is four rules in order: under core, spawn; room in the queue, enqueue; under max, spawn; otherwise it's full. That order contains a trap. The usual config is core 2, max 50, unbounded queue. I measured it: with 500 tasks needing ten concurrent threads, it used **two**. Not a bug – rule two says 'if there's room in the queue,' and an unbounded queue always has room, so rule three never runs. `max: 50` is dead config. p99 was **19,860 milliseconds** for a hundred-millisecond task, with **zero rejections** – green dashboard, twenty-second latency."

> "The fix is counterintuitive: make the queue *smaller*. Cap it at ten and the pool grows to eleven threads and p99 drops to 280 milliseconds – seventy times better, still zero rejections. **A full queue is the only signal that tells the pool to grow.** I'd also make that config throw in the constructor, because silently ignoring `max` is worse than failing loudly."

> "Second problem – the recursion one. Splitting a task into subtasks and submitting them is taught as good practice. But do the arithmetic: N threads, N tasks arrive, each takes a thread. Each submits a subtask, which queues. The subtask needs a thread; a thread frees when a parent finishes; a parent finishes when its subtask runs. That's a cycle. I measured it with eight threads: seven tasks complete fine. **Eight tasks complete zero, forever** – I ran the simulation to a hundred thousand milliseconds and not one counter moved. The code is identical at seven and eight; only the traffic changed, which means **no test will ever catch this** – tests run at low concurrency, and this only appears at peak."

> "Two fixes. The real one is not blocking on the subtask – that's what `thenApply` and `async/await` exist for, decoupling waiting from thread-holding. When you can't, use a **bulkhead**: give subtasks their own pool. With a separate pool, sixty-four tasks on eight threads all complete."

> "Third, what to do when it's genuinely full. There's no free option. I measured a producer running at 2.5× capacity. Unbounded queue: everything completes, but the queue hits 1,200 and p99 is **14.9 seconds** – that's a memory leak with a latency symptom. Reject: p99 drops to 690ms and memory is safe, but **57% of tasks are lost**. Caller-runs: everything completes, queue stays at 50, p99 is 680ms – the best row – **but the producer was blocked 59% of the total time.** If that producer is your HTTP acceptor, you stopped answering health checks and the load balancer just removed you. So the policy is a value the caller supplies; the pool shouldn't pick."

> "On sizing: `cores × (1 + wait / compute)`. I measured it three ways and it was exact each time – 4 for pure CPU, 8 for half-and-half, and **44** for a database-heavy workload. Same machine, same cores, eleven times the threads. And if those threads hit a database, the real limit is whichever is smaller: that number, or what the database can take concurrently."

> "For testing, five invariants, the important one being **every task has a destination** – completed, rejected, discarded, or handed back. 1,500 randomized runs, 847,000 tasks, zero violations. And I mutation-tested all four load-bearing lines; all four were caught. Making `shutdownNow` drop its queue instead of returning it failed 325 of 400 with 'submitted 757, accounted 746' – eleven tasks that vanished with no error at all. Without that accounting invariant, nothing would ever have noticed."

---

16. Follow-ups – work stealing, ప్రాధాన్యత, virtual threads

"ఒకే queue అన్ని threads కి – అది ఒక అడ్డంకి కాదా?"

అవును, ఎక్కువ cores ఉన్నప్పుడు. ప్రతి thread ఆ ఒక్క queue మీద lock కోసం పోటీ పడుతుంది. పరిష్కారం **work stealing**: ప్రతి thread కి సొంత queue, మరియు అది ఖాళీ అయితే ఇంకొకరి queue నుంచి **వెనక నుంచి** ఒకటి దొంగిలించడం (యజమాని ముందు నుంచి తీసుకుంటాడు, కాబట్టి ఘర్షణ తక్కువ). Java `ForkJoinPool`, Go యొక్క scheduler రెండూ ఇలాగే పని చేస్తాయి. ఒక అదనపు లాభం: ఉప-పని ఎక్కువసార్లు **అదే thread** మీద నడుస్తుంది, కాబట్టి §7 deadlock కొంతవరకు తగ్గుతుంది – కానీ **పూర్తిగా పోదు**, కాబట్టి bulkhead ఇంకా అవసరం.

"పనులకి ప్రాధాన్యతలు ఉంటే?"

`#q` ని ఒక priority queue చేయండి. కానీ అప్పుడు **starvation** వస్తుంది – తక్కువ ప్రాధాన్యత పనులు ఎప్పటికీ నడవకపోవచ్చు. DD 22 §11 lo `LevelQueue` మరియు DD 25 §16 lo ఇదే ప్రశ్న వచ్చింది; సమాధానం అదే: **ఎదురుచూసిన సమయాన్ని బట్టి ప్రాధాన్యత పెంచడం** (aging), లేదా ప్రతి స్థాయికీ ఒక వాటా ఇవ్వడం.

"Virtual threads (Java 21, Go goroutines) వచ్చాక ఇదంతా అవసరమా?"

మంచి ప్రశ్న, మరియు సమాధానం **సగం మాత్రమే**.

Virtual threads **§12 ని** పనికిరానిదిగా చేస్తాయి – అవి చాలా చౌక, కాబట్టి లక్షల కొద్దీ తయారు చేయొచ్చు, మరియు "ఎన్ని threads?" అనే ప్రశ్న అదృశ్యమవుతుంది. అవి **§7 deadlock ని కూడా** చాలావరకు పరిష్కరిస్తాయి – ఒక virtual thread ఎదురుచూస్తుంటే అది OS thread ని వదిలేస్తుంది, కాబట్టి వలయం తెగుతుంది.

కానీ అవి **§4 మరియు §10 ని ముట్టుకోవు**. మీ database ఇంకా 20 connections మాత్రమే భరిస్తుంది. మీ downstream API ఇంకా సెకనుకు 100 requests మాత్రమే. లక్ష virtual threads ఉన్నా, ఆ పరిమితులు అలాగే ఉంటాయి – కాబట్టి **మీకు ఇంకా ఒక limiter, ఒక bounded queue, మరియు నిండినప్పటి ఒక విధానం కావాలి**.

> **ఈ మార్పు ఏమి చెప్తోంది**
>
> Virtual threads **threads ఖరీదైనవి** అనే సమస్యని పరిష్కరిస్తాయి. అవి **మీ downstream పరిమితమైనది** అనే సమస్యని పరిష్కరించవు.
>
> మరియు ఈ రెండు docs (DD 24, DD 26) చెప్పేది ఏమిటంటే – **రెండోదే ఎప్పుడూ అసలు సమస్య**.

"ఒక పని exception విసిరితే?"

Thread చావకూడదు. ఒక పని ఇంకొక పనిని చంపకూడదు. Runner lo `try/catch` ఉండాలి, మరియు ఆ exception ని **ఒక చోటికి పంపాలి** (`onError` callback, లేదా `UncaughtExceptionHandler`). మౌనంగా మింగేస్తే – §13 నియమం 5 ప్రకారం – ఆ పని ఒక గమ్యం లేకుండా పోయినట్టే.

---

17. ఏమి నేర్చుకున్నాం

1. అపరిమిత queue ఒక దిండు కాదు, ఒక నోరు మూత. అది pool కి ఒత్తిడి సంకేతాన్ని ఎప్పటికీ చేరనివ్వదు, కాబట్టి `max` ఒక చచ్చిన సంఖ్య అవుతుంది. Queue ని **చిన్నది** చేయడం system ని **వేగం** చేసింది – 19,860 ms నుంచి 280 ms కి.

2. Thread pool తనను తానే అడగగలదు – అదే దాని ప్రత్యేకత. Connection pool కి ఈ సమస్య లేదు, ఎందుకంటే ఒక query ఇంకొక query ని అడగలేదు.

3. కొన్ని bugs నెమ్మదిగా రావు, ఒక్క అడుగులో వస్తాయి. 7 పనులు 100% పని చేశాయి; 8 పనులు 0% – శాశ్వతంగా. Code మారలేదు, traffic మారింది. అందుకే **ఇది test lo ఎప్పటికీ దొరకదు**.

4. నిండినప్పుడు ఉచిత ఎంపిక లేదు. తిరస్కరణ 57% పనిని పోగొట్టింది. Caller-runs ఏమీ పోగొట్టలేదు కానీ producer ని **59% సమయం ఆపేసింది**. అపరిమిత queue రెండూ చేయలేదు కానీ **memory ని తిని latency ని 14.9 సెకన్లు చేసింది**. మీరు ఏది భరించగలరో నిర్ణయించాలి – తప్పించుకోలేరు.

5. ఒకే machine మీద సరైన thread సంఖ్య 4 లేదా 44 కావచ్చు. అది cores మీద కాదు, **ఎదురుచూపు నిష్పత్తి** మీద ఆధారపడుతుంది. మరియు downstream ఒక పరిమితి పెడితే, అదే గెలుస్తుంది (DD 24 §12).

6. నా simulation నిజం కంటే దయగా ఉంది, మరియు అది ఒక bug ని దాచింది. పనిని inline గా నడపడం వల్ల deadlock **కనిపించలేదు** – 16 పనులు పూర్తయినట్టు చూపించింది. `after(0, ...)` చేర్చి, నిజమైన pool లాగా "submit వెంటనే తిరిగి వస్తుంది" అని చేశాక, అది **వెంటనే** కనిపించింది. **మీ model నిజం కంటే దయగా ఉంటే, అది మీకు అబద్ధం చెప్తుంది.**

7. "ప్రతి పనికీ ఒక గమ్యం" అనేది ఒక నియమంగా రాయదగినది. `submitted === completed + rejected + discarded + returned + queued + busy`. ఈ ఒక్క లెక్క `shutdownNow` మౌనంగా 11 పనులని మింగేస్తుందని బయటపెట్టింది – ఒక్క error, ఒక్క exception లేకుండా.

ఈ DOC నుంచి – మరియు ఈ మూడు DOCS నుంచి – ఒక్క వాక్యం

DD 24 lo pool **connections** ని పంచింది. DD 25 lo scheduler **సమయాన్ని** పంచింది. ఇక్కడ pool **నడిచే హక్కుని** పంచుతోంది.

మూడింటిలోనూ ఒకే నిర్మాణం కనిపించింది:

ఒక పరిమిత వనరు · దాని కోసం ఒక వరుస · ఆ వరుసకి ఒక పరిమితి · మరియు పరిమితి చేరినప్పుడు ఒక నిర్ణయం – అది వ్యవస్థది కాదు, దాన్ని వాడేవాడిది.

ఆ నాలుగో భాగమే ప్రతిసారీ మర్చిపోబడుతుంది. మరియు అదే ప్రతిసారీ రాత్రి 3 గంటలకు మిమ్మల్ని లేపుతుంది.